

Converging 4 resources

Recipe: apt::default

```
* execute[apt-get update] action run
- execute apt-get update
```

Recipe: cookbook1::default

```
* package[def-package] action install (up to date)
* service[service1] action enable
- enable service service[service1]

* service[service1] action start (up to date)
* cookbook_file[/root/magesh/cookbook1/www/index.html]
action create (up to date)
```

Chef Client finished, 2 resources updated

Please note that this synchronisation has to be done for each Chef agent, where we need to invoke the installation process.

Life cycle of the Chef server

Let us discuss the life cycle of the Chef server in the context of banking software being delivered in real-time. We will have two major processes in operation here, called continuous integration and continuous delivery.

Continuous integration (CI) is the process of taking the latest changes in the code check-in and doing an automated build with peripheral activities like a unit test, code coverage and static code analysis. This can be done with a suite of tools like Teamcity, Hudson or Jenkins. CI has multi-fold build options like incremental, nightly and weekly schedules. In the course of an incremental build, as soon as there is a code change, it is checked into the code repository. For a nightly build, a complete build is done, as against an incremental build, which only builds on recent changes. A weekly build does a complete build along with additional peripheral activities as listed above.

Once the CI is successful, the control of the process in the life cycle will be handed over to the continuous delivery (CD) process. CD is another process which runs through frameworks like Jenkins and is integrated with the Chef server environment. CD takes the build, which is successfully built by CI, and invokes the deployment process to the respective nodes (clients) invoked through Chef agents using the cookbooks, where the instructions for the deployment activity are fed. We can set the invoking script of CD in such a way that it happens only when CI execution is successful,

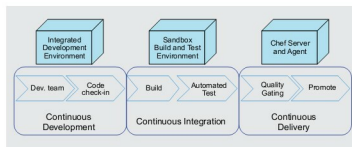


Figure 1: Chef server in continuous delivery (CD)

including or excluding peripheral activities. This means, we can invoke CD with build and test execution successful but some testcases failure or build and all testcases successful. Please note that CD will be invoked only when CI execution is successful, and then we can set the success factor to all unit test results being successful or only build being successful.

When we have a code repository linked to the CI server, which is in turn connected to the Chef server for CD, then whenever there is a code change, these automated processes of CI and CD will get triggered, and the nodes will have code packages installed and brought up automatically, using the recipe uploaded to the agent. This can be stopped till the unit integration test stage or the system integration test stage. Also, we can include recipe scripts to trigger automated test beds which run on their own in a typical black box environment. Once the test result is successful, the recipe can take the code build one stage further into the production test environment (a.k.a. user acceptance test).

A real-time DevOps Journey case study

A leading banking house in South Africa was running its IT development for more than a couple of decades on a legacy platform in a heterogeneous environment. This meant that it developed the front-end in AngularJS or Java, the back-end in a mainframe database, and service integration with mainframe systems which had undergone multiple releases over the past 20 years. There were also various supplier and feeder systems running on AIX, UNIX, Linux, etc. There was no control on development costs or on capacity expansion, which in turn cost the bank a lot.

This was studied by the CIO of the bank, and a team comprising business and technical consultants was set up to analyse and propose solutions on improving the IT platform to reduce residual costs incurred during capacity expansion. The CIO also wanted to improve the delivery time lines.

To focus on improving delivery time lines and to improve automated processes at each stage of the development, right up to the delivery stage, the consultant team suggested the formation of a task force which was referred to as the Chop Chop team. 'Chop Chop' means quickly or without delay. The key task of this team was to build a robust DevOps environment for the banking IT infrastructure to fit into the heterogeneous cluster of nodes (servers).

The key idea behind this cluster of nodes or server instances was to prepare the environment in such a way that...

- Clients were served by clustered server instances.
- Clustered instances could be automatically increased when load went up, with Chef recipes. A Chef monitoring tool would regularly monitor the load of the system and, depending on the increase, it could achieve resilience by doing a horizontal scaling of cluster nodes automatically using the Chef cookbook for capacity expansion.
- A dedicated cluster could be allocated to each region, if necessary, with standby Chef nodes with agents installed in them, which could be brought up when the need arose.